

TP4 - Sistemas Operativos

1. Problemas: Planificación de Procesos

1. Explique un esquema de planificación multinivel.

Un esquema de planificación multinivel funciona de manera tal que la cola de listos está particionada en varias colas separadas, con diferente prioridad para cada una. Cada una de estas colas puede tener su propio algoritmo de planificación (FCFS, SJF, RR) que no tengan las demás o sí. Lo que sí da igual cual algoritmo se escoja, siempre tendrá prioridad aquella con la cola de número más alto en prioridad. Esto da lugar a la inanición donde procesos de baja prioridad pueden no llegar a ejecutarse nunca. Para ello pueden haber soluciones como el envejecimiento de una cola donde se incrementa en el tiempo la prioridad de los procesos en espera.

Las colas multinivel generalmente en la práctica están separadas como dijimos con distintas prioridades, donde un ejemplo de prioridades es primero los procesos de tiempo-real, luego los procesos del sistema, los procesos interactivos y finalmente los procesos en lote. Un extra de las colas multinivel es que se puede optar por otro método de planificación distinto a la prioridad. Esto es dividir el tiempo entre las colas. Aquí, cada cola obtiene una cierta porción del tiempo de CPU, que luego puede programar entre sus diversos procesos. Por ejemplo, si tenemos una cola en primer plano y en segundo plano, a la cola en primer plano se le puede dar el 80 por ciento del tiempo de CPU para la programación de RR entre sus procesos, mientras que la cola en segundo plano recibe el 20 por ciento del tiempo de CPU para dársele a sus procesos sobre una base FCFS.

Y cómo última opción podemos tener una planificación multinivel realimentada, donde un proceso puede moverse entre varias colas de las presentes. En este caso hay que tener en cuenta que el planificador debe conocer el número de colas, los algoritmos de planificación para cada cola, el método usado para determinar cuándo mejorar un proceso, otro para cuando degradarlo, y el último para determinar en qué cola entra un proceso cuando necesita servicio.

2. ¿Cual es la ventaja de tener distintos quantum en los distintos niveles de un sistema de colas multinivel?

La ventajas de tener varios quantum en los distintos niveles es parecido a la opción nombrada arriba. Con esto nos podemos quitar uno de los problemas, que es la inanición ya que podemos hacer que un proceso que espere demasiado tiempo en una cola con menor prioridad pueda ser movido gradualmente a una cola de mayor prioridad. A continuación se explica cómo sucede esto.

Supongamos que tenemos un planificador multinivel retroalimentado, el cual tiene tres colas 1, 2 y 3. Lo primero que hará será ejecutar todos los procesos de la cola 1, y solo cuando esta esté vacía ejecutará los de la cola 2, y lo mismo para la 3. Como es apropiativo, si estoy ejecutando procesos de la cola 3 y llegan procesos a las colas de arriba se interrumpen los actuales para reemplazarlos por los de arriba.

Ahora bien, lo que pasa con los quantum el cual es más pequeño si la cola es más prioritaria, es que si llega un proceso a la cola 1, se le asigna a este un quantum de 8 milisegundos. Si en este tiempo no terminó de ejecutarse, lo movemos al final de la cola 2. Si ahora la cola 1 está vacía el proceso del frente de la cola 2 recibe un quantum de 16 milisegundos. Si nuevamente sucede que no llegue a completarse, es interrumpido y colocado al final de la cola 3. Finalmente cuando lleguen a la cola 3, los procesos de aquí serán atendidos con planificación FCFS, donde el primero en llegar es el primero en ser atendido. Estos solo se van a ejecutar cuando las colas de arriba 1 y 2 estén vacías. Y es así cómo sucede esto que nombramos al principio de cómo son movidos gradualmente los procesos para que ninguno quede en inanición. En conclusión, priorizamos entonces procesos cortos, pero sin olvidar los largos que serán eventualmente atendidos.

3. Relacione la planificación de colas multinivel y los diferentes quantum con la planificación tradicional de UNIX.

Antes de relacionar hay que poner en contexto un poco de como funciona la planificación tradicional de UNIX, donde lo que hace es aplicar una relación inversa entre los números de prioridad y las prioridades: cuanto mayor sea el número, menor será la prioridad. El planificador recalcula las prioridades de los procesos una vez por segundo utilizando la siguiente función:

$$\text{Prioridad} = (\text{uso reciente de CPU} / 2) + \text{base}$$

donde la base es un valor constante y el uso reciente de CPU se refiere a un valor que indica con qué frecuencia un proceso ha utilizado la CPU desde que se recalcularon las prioridades por última vez. Se puede ver mejor esto con un ejemplo donde para un proceso P1, el uso reciente de CPU es 40, para el de P2 18 y para P3 10. Si tenemos que base=60, entonces, si aplicamos la fórmula de prioridad:

Pr1 = 80, Pr2 = 69, Pr3 = 65. Donde podemos analizarlo de manera tal que un número de prioridad más alto significa una menor prioridad. Esto significa que el proceso P1, con una prioridad de 80, será el de menor prioridad relativa (será atendido después de los otros procesos). El proceso P3, con una prioridad de 65, tiene la mayor prioridad relativa (será atendido primero).

Entonces la relación que existe entre las colas multinivel y los diferentes quantum respecto a la planificación tradicional de UNIX recién nombrada, es que por un lado en ambos sistemas los procesos que usan más CPU tienden a ser penalizados, moviéndose a colas de menor prioridad o recibiendo un número de prioridad más alto en UNIX. Es decir que el manejo de prioridades es dinámico. Por otro lado, ambos logran resolver de esta manera el problema de la inanición sólo que si se diferencian en que como parámetro de referencia las multinivel utilizan el valor de los quantum, y la de UNIX el uso reciente de CPU.

CPU scheduling in UNIX is designed to benefit interactive processes. Processes are given small CPU time slices by a priority algorithm that reduces to round-robin scheduling for CPU-bound jobs.

Every process has a *scheduling priority* associated with it; larger numbers indicate lower priority. Processes doing disk I/O or other important tasks have priorities less than “pzero” and cannot be killed by signals. Ordinary user processes have positive priorities and thus are less likely to be run than is any system process, although user processes can set precedence over one another through the *nice* command.

The more CPU time a process accumulates, the lower (more positive) its priority becomes, and vice versa. This negative feedback in CPU scheduling makes it difficult for a single process to take all the CPU time. Process aging is employed to prevent starvation.

Página 21 de Apéndice C Silberschatz

4. ¿Es posible que ocurra la apropiación de un hilo (thread) mediante una interrupción de reloj? De ser así, ¿bajo qué circunstancias? Si no es así, ¿por qué no?

Depende del hilo con el que estemos trabajando, si el mismo es a nivel de usuario, entonces no pueden ser interrumpidos por el reloj a menos que se haya agotado la cantidad total de tiempo estimado para el mismo, es decir el valor del quantum asignado.

En cambio, para los hilos a nivel kernel sí que pueden ser interrumpidos individualmente en este caso por una interrupción de reloj, y las circunstancias que se tendrían que dar es que si este se está ejecutando durante demasiado tiempo, el reloj puede interrumpir el proceso y el hilo actual ya que justamente al ser un reloj puede contar la cantidad de tiempo que ha estado activo un proceso/hilo en el sistema.

5. En un sistema que soporte hilos (threads), ¿cómo son planificados estos frente al resto de los procesos?

Silberschatz 5.4 - Thread Scheduling

Hoy en día como dijo la profesora, los sistemas operativos trabajan con la planificación de hilos en vez de los procesos. Para los hilos a nivel de usuario, para planificarlos se necesita hacer uso de librerías, y el SO está inconsciente de ellos, mientras que los de nivel kernel si son planificados por el sistema operativo.

Si el sistema soporta hilos, hay que ver como es la relación entre los hilos del sistema operativo y los de usuario. Como hoy en día se ejecutan normalmente hilos de kernel en el procesador, y no de usuario, hay que mapear de alguna manera los hilos de usuario y de kernel. Para esto hay dos opciones: En sistemas muchos a uno y muchos a muchos, normalmente se mapean los hilos de usuario (manejados por una librería) a un proceso de peso ligero (LWP, light weight process). Esto se conoce como process contention scope (PCS) donde la competencia por la CPU se producen entre hilos que pertenecen al mismo proceso. Este LWP tiene su kernel thread, y este posteriormente tiene que ser programado para ejecutarse en el procesador.

Para decidir cual hilo a nivel kernel es el próximo a ser asignado a la CPU, el kernel usa un SCS (system-contention-scope) el cual hace que se compita por la CPU entre todos los hilos del sistema. Aquellos que adoptan una política uno a uno (como Windows o Linux), planifican los hilos utilizando solo SCS, ya que cada hilo de usuario es ejecutado directamente con su hilo de kernel, por lo que no hay necesidad de tener un PCS.

6. ¿Puede presentarse el problema de inversión de prioridades con hilos (threads) en el nivel de usuarios y a nivel de kernel? Justifique su respuesta.

Respuesta de Seba:

Como poder darse, se puede dar a nivel de kernel nomás, ya que el kernel ve hilos de kernel nomás, entonces para el mismo todos los hilos de usuario tiene la misma prioridad. Ahora, a nivel de kernel, sí podría darse.

Respuesta mía:

El problema de inversión de prioridad ocurre cuando un proceso de baja prioridad está en su región crítica y de repente un proceso de alta prioridad se vuelve listo y está programado. Si utiliza la espera activa, se ejecutará indefinidamente. Con los hilos de nivel de usuario, no puede suceder que un subproceso de baja prioridad sea reemplazado repentinamente para permitir la ejecución de un subproceso de alta prioridad. No hay reemplazo. Con los subprocesos de nivel de núcleo, este problema puede surgir.

7. Determinar el quantum q , es en general una tarea crítica. Asumamos que el tiempo de cambio de contexto es s y el tiempo promedio entre requerimientos de I/O para limitados por I/O es t . Discutir el efecto de cada una de las siguientes elecciones para q :

a) $q = \text{infinito}$

Si q es infinito entonces no tenemos restricciones de tiempo de ejecución para ningún proceso, pudiendo ejecutar desde el principio hasta el final sin cambiar el contexto. Esto es básicamente un sistema de planificación FCFS, el cual no es apropiativo ya que se ejecuta el primero que entra sin interrupciones, y sigue por orden de llegada.

El efecto no es muy bueno porque si tenemos un proceso que requiere esperar y bloquearse por necesitar varias I/O, sería ineficiente que se quede todo ese tiempo esperando consumiendo ciclos de CPU. Por otro lado, los procesos de larga duración monopolizarían el CPU, lo que puede llevar a una falta de respuesta rápida y problemas de inanición para otros procesos. En general, todo esto hace que aumente el **efecto convoy**.

b) $q = s$

Si q es igual al tiempo de cambio de contexto, entonces cada proceso se ejecutaría una pequeña cantidad de tiempo ya que si bien el cambio de contexto es costoso ($5 \mu s$) no es comparable a un valor de quantum promedio ($8000 \mu s$).

El efecto que tendría esto sería que la cantidad de cambios de contexto serían muchísimos más, perdiendo incluso más tiempo cambiando entre procesos que ejecutando el código de los mismos. La ineficiencia sería terrible, bajando mucho el rendimiento general.

c) $q = t$

Si q es igual al tiempo promedio entre operaciones de I/O para procesos I/O-bound (realizan muchas operaciones de entrada/salida.) lo que pasaría es que los procesos que están limitados por I/O se interrumpirán justo antes o después de solicitar una operación de I/O ya que es el promedio sin considerar ambos extremos.

El efecto no es realmente malo, ya que optimiza la ejecución para procesos I/O-bound. El sistema permite que dichos procesos completen sus ráfagas de CPU antes de entrar en espera por I/O, lo que reduce el tiempo de inactividad de la CPU mientras espera la finalización de la I/O.

d) q cerca de 0

Esto es casi lo mismo que $q = s$, e incluso peor ya que sería hasta más pequeño el tiempo de ejecución. De más está decir que no es una buena elección, ya que prácticamente no hay progreso significativo en la ejecución de los procesos antes de que se realicen cambios de contexto.

e) $s < q < t$

Lo que pasaría aquí, es que el quantum es mayor que el tiempo de cambio de contexto pero menor que el tiempo promedio entre solicitudes de I/O. A simple vista, parece ser una elección razonable y equilibrada. No habría tantos cambios de contexto, permitiendo que los procesos se ejecuten en un tiempo considerable antes de ser interrumpidos. Al ser $q < t$, los procesos I/O-bound podrían completar sus ráfagas de CPU y entrar a esperar por I/O, manteniendo a la CPU ocupada con otros procesos mientras tanto. Los procesos CPU-bound también recibirían algo de tiempo de CPU, aunque serán interrumpidos antes de que terminen, pero sin un impacto significativo en el rendimiento.

f) $q > t$

En estas condiciones, lo más probable es que los procesos I/O-bound soliciten una I/O antes de que se agote el quantum por el valor que se le da. Si estamos con procesos I/O-bound es una buena opción, ya que no interrumpiría el proceso innecesariamente, ya que probablemente solicitará una operación I/O antes de que finalice el quantum.

Por otro lado, para procesos CPU-bound puede generar una latencia mayor, ya que la CPU podría estar ocupada con un proceso por un largo tiempo antes de ceder la CPU a otros procesos. Esto podría aumentar los tiempos de respuesta y causar problemas de equidad entre los procesos.

43. The CPU efficiency is the useful CPU time divided by the total CPU time. When $Q \geq T$, the basic cycle is for the process to run for T and undergo a process switch for S . Thus, (a) and (b) have an efficiency of $T/(S + T)$. When the quantum is shorter than T , each run of T will require T/Q process switches, wasting a time ST/Q . The efficiency here is then

$$\frac{T}{T + ST/Q}$$

which reduces to $Q/(Q + S)$, which is the answer to (c). For (d), we just substitute Q for S and find that the efficiency is 50%. Finally, for (e), as $Q \rightarrow 0$ the efficiency goes to 0.

8. Se ha visto en teoría un conjunto de políticas de planificación y un conjunto de mecanismos para implementar estas políticas. Algunos objetivos de algoritmo de planificación son:

- a) Maximizar la terminación de tareas por unidad de tiempo.
- b) Maximizar el número de usuarios interactivos recibiendo respuesta en un tiempo aceptable.
- c) Ser predecible.
- d) Minimizar la sobrecarga.
- e) Balancear la utilización de recursos.
- f) Evitar tareas pospuestas indefinidamente.
- g) Ordenar las tareas en función de su importancia y requerimientos de eficiencia.
- h) Favorecer procesos que mantienen recursos claves no apropiables.
- i) Reducir el servicio cuando la carga resulta excesiva.

TERMINAR

¿Cuál de los objetivos anteriores se debe tener en cuenta cuando ...?

i. Ante un usuario que ha esperado durante una excesiva cantidad de tiempo se lo favorece.

e -

ii. Un proceso arriba, pero no puede ejecutarse por que necesita de un recurso que este alocado a un proceso de menor importancia.

g-

iii. Durante períodos picos no alcanzar un colapso provocado por la sobrecarga de mantener un gran número de procesos.

iv. Los procesos compitiendo por la CPU se complementan en sus requerimientos de recursos.

e - f - h

v. Un sistema de control de procesos en tiempo real monitoreando a una estación de servicios requiere una rápida respuesta.

vi. A un proceso que está esperando determinado tiempo se le incrementa su prioridad para acelerar su atención.

e-

9. Defina las diferencias entre planificación con apropiación y sin apropiación. ¿Por qué un algoritmo sin apropiación no es comúnmente utilizado en centros de cómputos?

La principal diferencia es que por un lado en una planificación apropiativa el SO puede interrumpir un proceso que está ejecutándose para asignar la CPU a otro proceso, incluso si el proceso actual aún no ha terminado o no está esperando I/O. Mientras que en uno no apropiativo, una vez que un proceso tiene el control de la CPU no puede ser interrumpido hasta que termine su ejecución o pase a un estado de espera por un I/O. Si llegara a pasar, no lo hace el sistema sino que el mismo proceso voluntariamente.

Entonces un algoritmo sin apropiación no es comúnmente utilizado en los centros de cómputo por las razones que venimos comentando anteriormente. Porque un proceso con tiempo de ejecución muy largo puede monopolizar toda la CPU, aumentan el riesgo de deadlocks, desaprovechan recursos al no poder reasignarlos si un proceso no los usa de manera eficiente, y tienen tiempos de respuesta variables ya que los procesos no pueden ser interrumpidos, lo que afecta la flexibilidad y la eficiencia del sistema.

10. Suponga un algoritmo de planificación de corto plazo que favorece a aquellos procesos que han usado poco tiempo de procesador en el más reciente pasado (no solo en el último quantum). ¿Por qué este algoritmo favorece a programas limitados por I/O y no posterga permanentemente programas limitados por la CPU?

Este algoritmo favorece a programas limitados por I/O de una gran manera, ya que estos programas tienden a ejecutar rafagas cortas en el cpu (ya que están muy limitados por operaciones de I/O), por lo que si en general usan poco tiempo de cpu, por este algoritmo se verían beneficiados.

Si uno ve esto, tendería a pensar que los procesos limitados por cpu quedarían constantemente relegados (ya que usan rafagas largas de cpu). Sin embargo, como el algoritmo se basa en el reciente pasado, ya pasado un tiempo estos procesos van a no tener historia, por lo que no van a tener nada con lo que ser comparado, y entonces serán ejecutados en algún momento (ya que al no tener historia, necesitan tenerla para compararla con el resto de procesos).

11. Una variante del algoritmo de Round Robin es retornar el proceso que ha agotado su quantum al final de la cola, el que ha utilizado la mitad del quantum en el medio de la cola (luego de atender la entrada y salida) y así sucesivamente. Compare esta alternativa con Round Robin.

Una ventaja de este método es que permite tiempos de respuesta más rápidos para los procesos que utilizan menos tiempo de CPU, recompensando a aquellos que están limitados por E/S o que liberan la CPU voluntariamente. Por otro lado, una desventaja de esta política de programación es la complejidad adicional en el mantenimiento de la cola de listos y el potencial de tiempos de espera más largos para los procesos limitados por la CPU, lo que podría conducir a un aumento de los tiempos de espera promedio y de los tiempos de respuesta.

Además, esta política puede tener implicaciones para la asignación de recursos y el rendimiento del sistema. La asignación de procesos de esta manera diferenciada puede resultar útil en situaciones en las que es necesario gestionar de manera eficiente una combinación de tareas cortas y largas, lo que podría mejorar el rendimiento general del sistema para determinadas cargas de trabajo. Sin embargo, también podría generar injusticias o inutilización de determinados procesos si no se gestiona correctamente.

12. La mayoría de los planificadores de tipo Round Robin utilizan un quantum de tamaño fijo. De un argumento en favor de un quantum pequeño y uno en favor de uno grande.

A favor de uno grande: Al tener un quantum mas grande, los procesos están más tiempo en el procesador ejecutándose, por lo que podría ser que haya más cantidad de procesos completados (y una posible reducción del tiempo de retorno), además de que al haber menos cambios de contexto, el tiempo gastado en estos se reduce.

A favor de uno pequeño: Se reduce el efecto convoy, ya que se le da tiempo a cada proceso de ejecutar un poco. Además, se reduce el tiempo de respuesta, lo que en sistemas de tiempo real puede ser muy útil.

Un poco más:

Si el tiempo de cambio de contexto es grande, entonces el valor del quantum de tiempo debe ser proporcionalmente grande. De lo contrario, la sobrecarga del cambio de contexto puede ser bastante alta. La elección de valores de quantum de tiempo grandes puede conducir a un sistema ineficiente si los tiempos de ráfaga típicos de la CPU son menores que el quantum de tiempo. Si el cambio de contexto es muy pequeño o insignificante, entonces el valor del quantum de tiempo se puede elegir con más libertad.

13. Sea una entidad bancaria que atiende a sus clientes a través de cajeros automáticos y en las ventanillas de sus sucursales. Los cajeros automáticos y las terminales de las ventanillas están conectadas al sistema computador central y además, en este sistema se procesan tareas de tipo batch que utilizan cintas magnéticas, impresoras y discos. Se desea priorizar por sobre todas las tareas a las tareas batch, luego con menor prioridad a los cajeros automáticos y por último las tareas de las terminales.

a) Diseñe una política de administración del procesador que logre este cometido y provea un balance equitativo de los recursos.

b) Indique la política de administración de cada cola de listos.

14. Suponga que arriban nuevos procesos al sistema a una media de seis procesos por minuto y que cada uno de ellos precisa de una media de 8 segundos de tiempo de servicio. Estime la fracción de tiempo en que la CPU está ocupada en un sistema monoprocesador.

Número de procesos por minuto = 6

Tiempo de ráfaga de cada proceso = 8 segundos

Tiempo de utilización de la CPU en un minuto = $6 \times 8 = 48$ segundos

% de utilización de la CPU = $\text{tiempo útil} / \text{tiempo total} \times 100 = (48/60) \times 100 = 80\%$

2. Problemas: Procesos y Planificación de Procesos

1. Considere el siguiente conjunto de procesos, estando la duración de las ráfagas de CPU especificada en milisegundos:

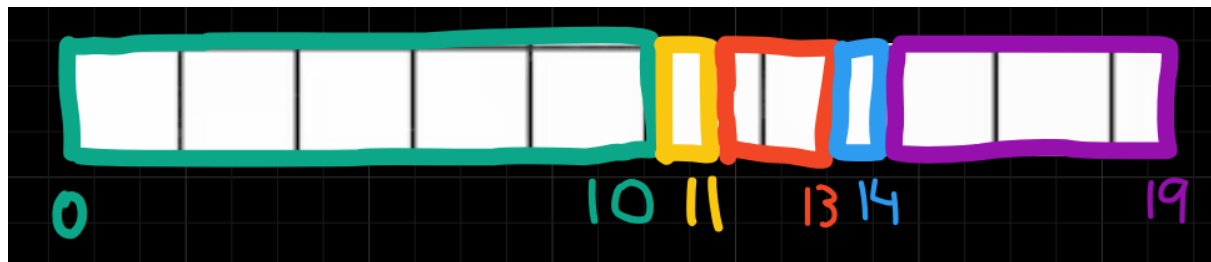
Proceso	Tiempo de Ejecución	Prioridad
P ₁	10	3
P ₂	1	1
P ₃	2	3
P ₄	1	4
P ₅	5	2

Suponga que los procesos llegan en el orden P1, P2, P3, P4, P5 y se encuentran listos en el tiempo 0

a,b,c) Muestre la ejecución de estos procesos utilizando el diagrama de Gantt para los siguientes algoritmos de planificación: FCFS, SJF, planificación por prioridad no apropiativa (un número de prioridad baja indica una prioridad alta) y round robin con quantum = 1. Dar tiempo de retorno de cada proceso, tiempo de retorno promedio para cada uno de los algoritmos y el tiempo de espera.

P1 - P2 - P3 - P4 - P5

FCFS



Tiempo de retorno:

P1 = 10 - P2 = 11 - P3 = 13 - P4 = 14 - P5 = 19

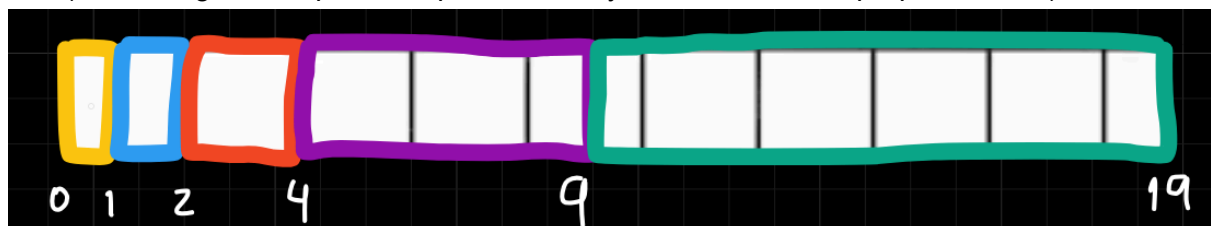
Tiempo de retorno promedio:

13,4

Tiempo de espera: 9.6 prom

P1 = 0 - P2 = 10 - P3 = 11 - P4 = 13 - P5 = 14

SJF (como tengo 0 tiempo de respuesta no hay diferencia entre apropiativo o no)



Tiempo de retorno:

P1 = 19 - P2 = 1 - P3 = 4 - P4 = 2 - P5 = 9

- b) ¿Cuál es el tiempo de retorno de cada proceso para cada algoritmo de planificación planteado en el inciso anterior? ¿Cuál es el tiempo de retorno promedio para cada uno de los algoritmos planteados?
- c) ¿Cuál es el tiempo de espera?

3. Dado el diagrama transición de procesos de la figura 1, que amplía y completa al anterior: Se pide:



a) Indicar qué provoca las transiciones 1 a 8. Que rutinas intervienen y cuando corresponda, qué interrupciones las inician.

Transición 1: El programa es activado y se crea el proceso, el planificador a largo término le asigna un lugar en memoria, y es puesto en la cola de listos (dependiendo de cómo sea el planificador, variarán las colas, por ejemplo, si es cola multinivel, si le asigna una).

Transición 2: El programa es seleccionado para ejecutar, y se le da su tiempo, acá intervendrá el planificador a corto término.

Transición 3: El proceso, que estaba en ejecución, es interrumpido por el SO para dejar que ejecute otro proceso. Ocurre el cambio de contexto (se guarda todos los valores de los registros, y su estado actual).

Transición 4: El proceso requiere un I/O, en este caso del disco, realiza una interrupción y el SO lo manda a este nuevo estado, el proceso deja de ejecutarse.

Transición 5: El proceso pudo realizar su operación de I/O, y es llevado de nuevo a la cola de listo, esperando para ejecutarse de vuelta.

Transición 6: El proceso requiere un I/O, en este caso del disco, realiza una interrupción y el SO lo manda a este nuevo estado, el proceso deja de ejecutarse.

Transición 7: El proceso pudo realizar su operación de I/O, y es llevado de nuevo a la cola de listo, esperando para ejecutarse de vuelta.

Transición 8: El proceso terminó de realizar sus tareas, y termina su ejecución

b) Supongamos que el sistema ejecuta 2 procesos de las siguientes características:

PROCESO 1 : Ejecuta 30 ms., efectúa una E/S sobre cinta, ejecuta 10 ms. y termina.

PROCESO 2 : Ejecuta 10 ms., efectúa una E/S sobre cinta, ejecuta 10 ms., efectúa una E/S sobre disco, ejecuta 10 ms. y termina.

(*) Tiempo empleado por el Sistema Operativo para tomar los 2 procesos a comenzar y colocarlos en la cola de Listos. Luego la rutina 7 coloca el Proceso Nro. 1 en estado de ejecución.

Adem´as se supone :

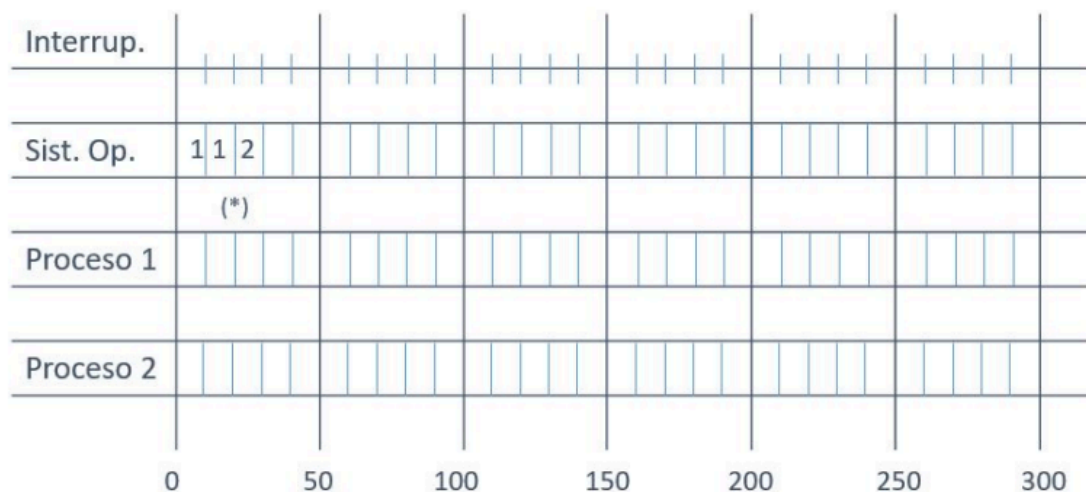
Las rutinas 1 a 8 ejecutan 10 ms. ante cualquier evento.

El método de selección de la cola de listos es el FIFO, asignándole a cada proceso 20 ms.

El sistema tiene 2 canales (disco y cinta) administrados por semáforos.

Una operación de E/S sobre cinta tarda 70 ms. y sobre disco 40 ms.

Se pide completar el diagrama de la Figura 2.



	000	010	020	030	040	050	060	070	080	090	100	110	120	130	140	150	160	170	180	190	200	210	220	230	240	250	260	270	280	290	300
Interrupción																															
Sist op	1	1	2			3	2		6	2		6					7		2	7	4	2		8		5		2		8	
Proceso 1	L	L	-	E	E	-	L	L	L	-	E	-	BC	BC	BC	BC	BC	BC	BC	-	L	-	E	-	T						
Proceso 2	-	L	L	L	L	L	-	E	-	BC	BC	BC	BC	BC	BC	BC	-	L	-	E	-	BD	BD	BD	BD	-	L	-	E	-	T

c) ¿Qué modificaciones le realizaría al inciso b) si se tuviera en el sistema 2 procesadores?

Se podrían ejecutar ambos procesos paralelamente, y podrían hacerse dos rutinas del SO al mismo tiempo (?), esto último habría que consultarlo. Pero en caso de que si, ambos procesos comenzarán a ejecutarse al mismo tiempo, e ir manejando independientemente uno del otro, ya que cada uno tendría su procesador.

d) Cómo modificaría el diagrama de transición de procesos si se previera el uso de semáforos para sincronizar los procesos entre sí. Que nuevos estados habría que agregar al diagrama de transiciones y que nuevas rutinas de manejo de las mismas habría que prever si:

1) Se incluye un semáforo que controla el uso del canal correspondiente a las unidades de cinta y otro al correspondiente a las unidades de disco.

2) Se incluye un semáforo que controla la obtención dinámica de espacio, en memoria central por parte de los procesos en ejecución

4. Supóngase un sistema operativo que sigue un modelo cliente-servidor (o sea, con arquitectura de micronúcleo) en el que existe un proceso servidor FS (File system) encargado de la gestión de archivos y un proceso TD (Tarea de Disco) que realiza la función de manejador de disco. En este sistema, la prioridad de FS es mayor que la de un proceso de usuario y la de TD mayor que la del resto de procesos, incluyendo FS.

Considere los siguientes procesos de usuario A, B y C y cada uno con las siguientes características:

Proceso A: 160 ms de CPU, 50 ms de E/S a disco, 50 ms de CPU

Proceso B: 20 ms de CPU, 50 ms de E/S a disco, 60 ms de CPU

Proceso C: 160 ms de CPU, 50 ms de E/S a disco, 50 ms de CPU

Se solicita construir un diagrama de tiempos donde se muestre los estados de los diferentes procesos. Considere las siguientes situaciones:

a) Los procesos A, B y C arriban juntos y se ubican en la cola de listos en este orden.

b) Los procesos A y B arriban juntos y tienen este orden en la cola de listos y el proceso C arriba 20 ms después.

Considere que los procesos FS y TD requieren 20 ms de CPU, las rutinas correspondiente al kernel requieren 10 ms y tiene puntos de apropiación cada 10 ms. Los procesos de usuario de este sistema operativo se planifican a intervalos de 100 ms.

Calcule para cada uno de los procesos A, B y C el tiempo de retorno y de espera.

5. Considerando que en un determinado instante de tiempo el sistema tiene el siguiente conjunto de procesos:

a) P1: 30 ms de CPU, 50 ms de disco, 45 ms de CPU, 50 ms de disco y 15 ms de CPU.

b) P2: 45 ms de CPU, 50 ms de disco y 25 ms de CPU.

c) P3: 35 ms de CPU, 80 ms de cinta y 30 ms de CPU.

d) P4: 20 ms de CPU, 80 ms de cinta, 150 ms de CPU, 50 ms de disco y 15 ms de CPU.

Se utiliza un algoritmo de planificación RR con un quantum de 20ms y presenta puntos de apropiación cada 5 unidades de tiempo. Todos los procesos ingresan en el tiempo t. Cada rutina del S.O. Se ejecuta en 5ms.



(a) Realice el diagrama de estado de procesos.

(b) ¿Cuál es el estado de las colas correspondientes al instante de tiempo $t+100\text{ms}$ y en el instante de tiempo $t+160\text{ms}$?

(c) Especifique el tiempo de retorno para cada uno de los procesos.

(d) Especifique el tiempo de espera para cada uno de los procesos.

(e) ¿Cuál sería la diferencia si el algoritmo de planificación fuera de colas multinivel, teniendo 2 niveles de colas, inicialmente todos los procesos ingresan en la cola de mayor prioridad, nivel 1, si utilizan completamente su quantum se lo envía a la cola del nivel 2. Para esta situación, ¿el estado de las colas es el mismo? ¿y los tiempos de espera?

10)